DF Daniil Fedorov
March 20 2025 | Updated January 19 2026

★ ★ ★ ★ ★

How to Create and Manage a Python 3 Virtual Envir on Ubuntu 22.04

Linux Ubuntu

Introduction

Python is one of the most popular programming languages for backend development, automation, data analysis, and DevOps tasks. On Linux serv run scripts, web applications, and background services.

In this guide, you will learn how to create and manage a Python 3 virtual environment on Ubuntu 22.04. Virtual environments allow you to isolate pro version conflicts, and keep your system Python installation clean.

Preparing for installation

This tutorial is suitable for Ubuntu 22.04 Server and VPS environments. All commands are executed using sudo privileges.

Before installing the packages, you need to follow our guide to running Ubuntu Server 22.04 as a standard user.

Installing Python 3 and pip on Ubuntu 22.04

Let's update the package index and run the command to update the packages to the latest releases:

```
sudo apt update && sudo apt upgrade -y
```

The -y flag automatically confirms all prompts during the upgrade process.
Checking the Python version goes like this:

```
python3 --version
```

Output is going to be like this:

```
#Output
Python 3.10.6
```

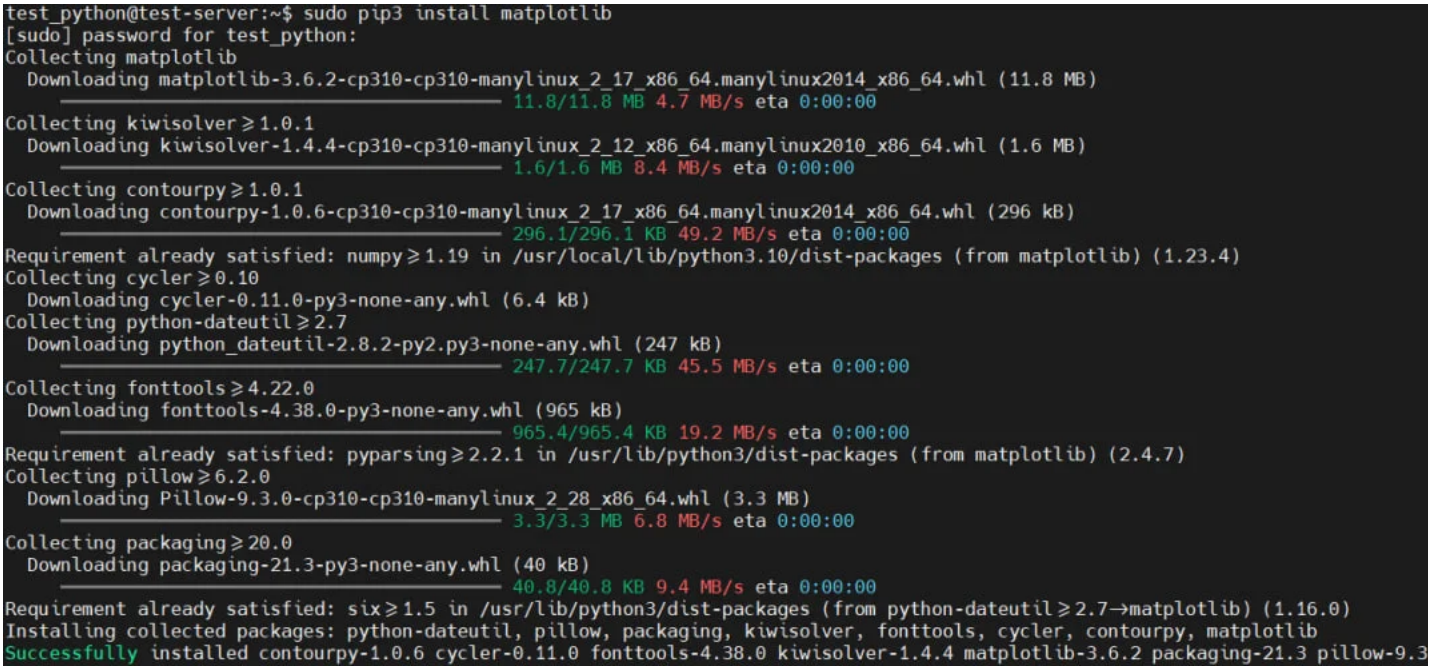
The next step is to install python3-pip in order to manage Python packages. Let's use the built-in command:

```
sudo apt install python3-pip -y
```

To install the matplotlib library, you must run the following command and the result is shown in Screen 1:

```
pip3 install matplotlib
```

Installing Python 3 and pip on Ubuntu 22.04



Screen 1 -Installing the matplotlib library

To make sure the software environment is reliable, you need to install several packages

```
sudo apt install build-essential libssl-dev libffi-dev python3-dev
```

The first stage has been completed. We have updated the package index and updated obsolete packages, the current version of the pip3 packag
installed.

Setting up a virtual environment

A Python virtual environment is an isolated workspace that contains its own Python binaries and installed libraries. Using virtual environments on U
dependency conflicts between different projects and system-level Python packages.
The virtual environment is deployed using the installed venv (virtual environment) package:

```
sudo apt install python3-venv -y
```

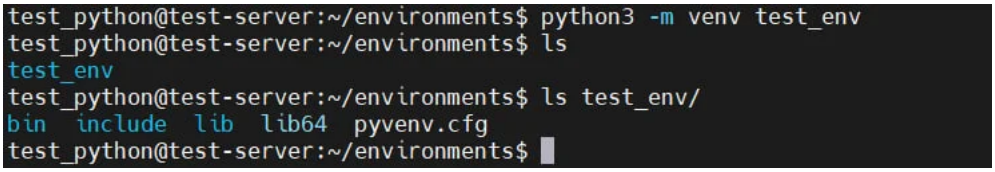
Then let's create a directory called test:

```
mkdir test
cd test
```

Change to the first directory and use the following command to create a virtual environment called test_env:

```
python3 -m venv test_env
```

The result is shown in Screen 2.



Screen 2 - Create a virtual environment

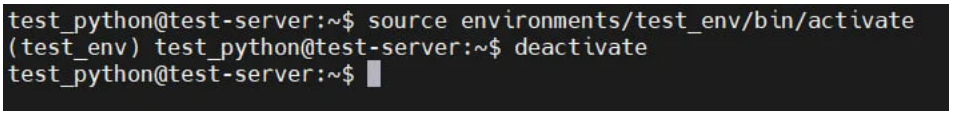
The generated files configure the virtual environment to work separately from our host files. Activation of the environment is as follows, and to dis must run the deactivate command:

```
source test_env/bin/activate
```

To disable the virtual environment, run the command:

```
deactivate
```

The results are shown in Screen 3.



Screen 3 - Activating and deactivating a virtual environment

In the figure, you can see that after launch, an inscription appears in front of the user name (test_env) indicating that all commands are executed the next step is to consider running a regular code written in the Python programming language.

Testing the virtual environment

After activation, you need to create a file with the extension .py:

```
vim thanks.py
```

И вставим следующий кусок кода:

```
print("Dear User,\n"
"Thank you for using tutorials from \n"
"Serverspace Team")
```

To run the program, do the following:

```
python3 thanks.py
```

And we get the following result, as shown in Screen 4.

```
(test_env) test_python@test-server:~$ vim thanks.py
(test_env) test_python@test-server:~$ python3 thanks.py
Dear User,
Thank you for using tutorials from
Serverspace Team
(test_env) test_python@test-server:~$
```

Screen 4 - Running code in a virtual environment

At this point, the stage ends and in order to complete the process of working in the virtual environment, we will execute the “deactivate” commar environment.

Conclusions

In this guide, you learned how to install Python 3 and pip on Ubuntu 22.04, create and manage a Python virtual environment, and test isolated proj environments are essential for maintaining clean dependencies and stable Python development on Linux servers.

FAQ (Frequently Asked Questions)

- **How do I check which version of Python is installed on Ubuntu 22.04?**
  A virtual environment isolates project dependencies, prevents version conflicts, and allows multiple Python projects to run independently on
- **Why do I need a virtual environment in Python?**
  A virtual environment isolates project dependencies, preventing conflicts between different Python libraries and ensuring stable developmer
- **What is the difference between pip3 and venv?**
  pip3 is a package manager used to install and manage Python libraries, while venv creates isolated environments where these libraries are st
- **How do I activate and deactivate a Python virtual environment?**
  To activate: source project_env/bin/activate
  To deactivate: deactivate
- **Can I have multiple virtual environments on the same server?**
  Yes, you can create as many as needed, each in its own directory, with its own dependencies.